

Gypsum Belt-Filter Cake Recognition

Automated Valid / Invalid Detection from a Plant Camera Feed

Final Project Report

AI Expert — Final Project Course

Yaron Conroy

2026-09-23

Code repository: github.com/yaronconroy-del/picture-recognition-gypsum

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Introduction	5
2.1 Problem Statement	5
2.2 Background and Motivation	5
2.3 Related Work.....	5
3. Dataset	7
3.1 Class Distribution.....	7
3.2 Sample Images	7
3.3 Known Dataset Characteristics	8
3.4 The Day/Night Anomaly	8
3.5 Exploratory Data Analysis.....	9
4. Methodology.....	10
4.1 Preprocessing	10
4.2 Leakage-Safe Train/Test Splitting	10
4.3 Model Architecture and Fine-Tuning Strategy.....	10
4.4 Handling Class Imbalance	11
4.5 Training Configuration.....	11
5. Experiments and Results	12
5.1 Stage 1 — Baseline: Lite vs. Extended Label Scheme	12
5.2 Stage 2 — No-Empty Scheme (Round 2)	12
5.3 Label-Scheme Decision.....	14
5.4 Stage 3 — Threshold, ROC, PR, and Calibration Analysis.....	14
5.5 Stage 4 — Ensemble, Test-Time Augmentation, and Calibration	15
5.6 Stage 5 — Live Demo Scripts	16
6. Decision Log	18
7. Business Case and Value Estimate.....	19
7.1 Yield Loss	19
7.2 Downtime (Equipment Damage)	19
7.3 Combined Value Estimate	19
8. Conclusions	20
9. Limitations.....	20
10. Further Directions	21
10.1 Near-Term	21

10.2 Full-Scale Pilot Roadmap	21
11. Appendix	22
A. Repository Structure	22
B. References.....	22

1. Executive Summary

This project builds an image classifier that looks at a single camera frame of a gypsum belt filter — part of a wet-process phosphoric acid production line — and automatically determines whether the filter cake is VALID (properly dewatered) or INVALID (wet gypsum), so that filtration problems can be caught automatically instead of relying solely on intermittent manual inspection.

Wet gypsum causes two measurable costs: lost, unrecovered phosphoric acid (yield loss) and downstream equipment damage (downtime). Automating detection instead of relying on manual checks is estimated to recover roughly \$1.48M–\$2.21M per year, at a 10–15% recovery rate against a combined annual exposure of approximately \$14.76M — the full calculation, with every assumption stated explicitly, is in Section 7.

Working from a small dataset of 176 labeled photographs, the project systematically compared four label schemes, three train/test split ratios, cross-validation, class-imbalance handling, decision-threshold tuning, and ensemble techniques — 22 individually reasoned experiments in total, each building on the last. The best-performing and operationally simplest model, a 4-class scheme that folds “empty filter” into “invalid” (no-empty split), was selected based on converging statistical evidence (cross-validation accuracy, ROC AUC, PR AP) and an independent operational argument: a deployed model does not need to recognize “empty” at all, since the operator empties the filter themselves and already knows it.

The project also produced two working local scripts — a single-photo classifier and a simulated live-feed watcher — and an honest accounting of what remains open: the promoted model has not yet been trained on the full dataset for a final, non-abbreviated run; the “day”/“night” labels do not actually correspond to time of day, and their true meaning is still unresolved; and a real camera/alerting integration remains to be built. These, and a roadmap for a full-scale plant pilot, are set out in Sections 9 and 10.

2. Introduction

2.1 Problem Statement

A fixed industrial CCTV-style camera is mounted above a gypsum belt filter on a phosphoric-acid production line. Filtration quality is currently checked by manual visual inspection, which is inherently intermittent (an operator cannot watch the feed continuously) and inconsistent (judgment varies between operators and shifts). The goal of this project is an image classifier that looks at a single camera frame and determines whether the cake is VALID or INVALID, automatically and continuously — closing the gap between when a problem happens and when it is noticed.

This is a proxy for a real yield-loss and equipment-damage problem, not a cosmetic visual check: the visual cracking/patchiness that marks a cake as “invalid” is how wet, poorly-dewatered gypsum looks on camera.

2.2 Background and Motivation

This project comes directly from a real process at the author's plant, a wet-process phosphoric acid producer: phosphate rock reacts with sulfuric acid to produce phosphoric acid, with gypsum as a by-product. The gypsum still carries residual phosphoric acid, so it is washed with water and then dewatered on the belt filter this project's camera watches, to recover as much phosphoric acid as economically possible before the gypsum is discarded.

“Invalid” means wet gypsum — the cake did not dewater properly. This has two separate costs:

1. Yield loss — phosphoric acid trapped in gypsum that is still wet is lost with the discarded gypsum. Every 1% of phosphoric acid remaining in the gypsum costs roughly \$2,000/hour. On an annual basis this typically runs to a loss equivalent to about 3% of throughput, with bad-filtration episodes accounting for roughly 6% of downtime.
2. Equipment damage / downtime — wet gypsum can damage downstream equipment, adding a further ~3% of annual downtime on top of the yield-loss figure above.

The full monetized business case, with every assumption stated explicitly, is worked through in Section 7.

2.3 Related Work

A short survey of existing approaches to similar problems was carried out to check this project's plan against established practice rather than reinventing it from scratch. No published work was found on this exact equipment (gypsum belt filter cake inspection specifically); the closest matches are the general “surface/assembly defect via transfer-learned CNN” literature summarized below, which is why this project's approach follows that pattern.

- Transfer learning from ImageNet-pretrained CNNs is the standard approach for small industrial visual-inspection datasets. A 2024 systematic review of CNN-based surface-defect detection found transfer learning used in 83% of studies surveyed, specifically because training a CNN from scratch needs a dataset far larger than most factories can realistically label. This directly matches this project's choice: MobileNetV2, transfer-learned rather than trained from scratch, on a 176-image dataset that would never support training a CNN from zero.
- Class imbalance between “normal” and “defect” is treated as the norm, not the exception, in industrial defect detection, because in a working process defects genuinely are rarer than normal

output. This project's “empty filter” class (8 of 176 images, 4.5%) sits in the same range as examples cited in the literature, and the standard mitigations the literature points to — cost-sensitive/class-weighted loss and resampling — are exactly what this project tried, including the honest negative finding that resampling added nothing once weighted loss was already in place.

- Reported accuracy in comparable published work is high (often 95%+), but on datasets one to three orders of magnitude larger than this one — e.g. a transfer-learning assembly-defect inspection system reaching 98.67% accuracy was trained on thousands of labeled images per station, not 176 total across 4–5 classes. This project's own numbers (~77% cross-validation accuracy on the leading scheme) should be read against that gap, not against the field's best-case numbers.

References are listed in Appendix B.

3. Dataset

The dataset consists of 176 photographs captured by the plant's fixed camera and originally exported via WhatsApp (filenames followed the WhatsApp Image <date> at <time> (<n>).jpeg pattern). They were manually sorted into folders that act as class labels, then renamed to random IDs (img_<id>.jpeg) with a manifest mapping back to the originals, so filenames cannot leak label information.

3.1 Class Distribution

Class	Meaning	Condition	Time of Day	# Images
תקין יום	Valid — day	Normal/OK cake	Day	62
תקין לילה	Valid — night	Normal/OK cake	Night	30
לא תקין יום	Invalid — day	Abnormal/faulty cake	Day	42
לא תקין לילה	Invalid — night	Abnormal/faulty cake	Night	34
מסנן ריק	Empty filter	No cake on filter	N/A	8

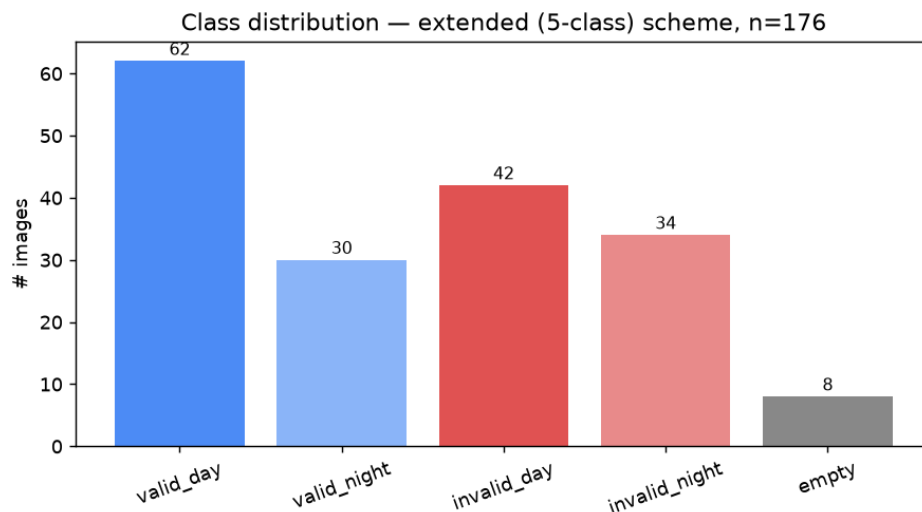


Figure 1. Class distribution, extended (5-class) scheme, $n = 176$.

3.2 Sample Images

Figure 2 shows four random examples per class. The burned-in timestamp overlay (top-right) and the night floodlight glare are both visible; note that at least one “valid, day” sample also shows a bright glare spot, which foreshadows the day/night finding in Section 3.4.

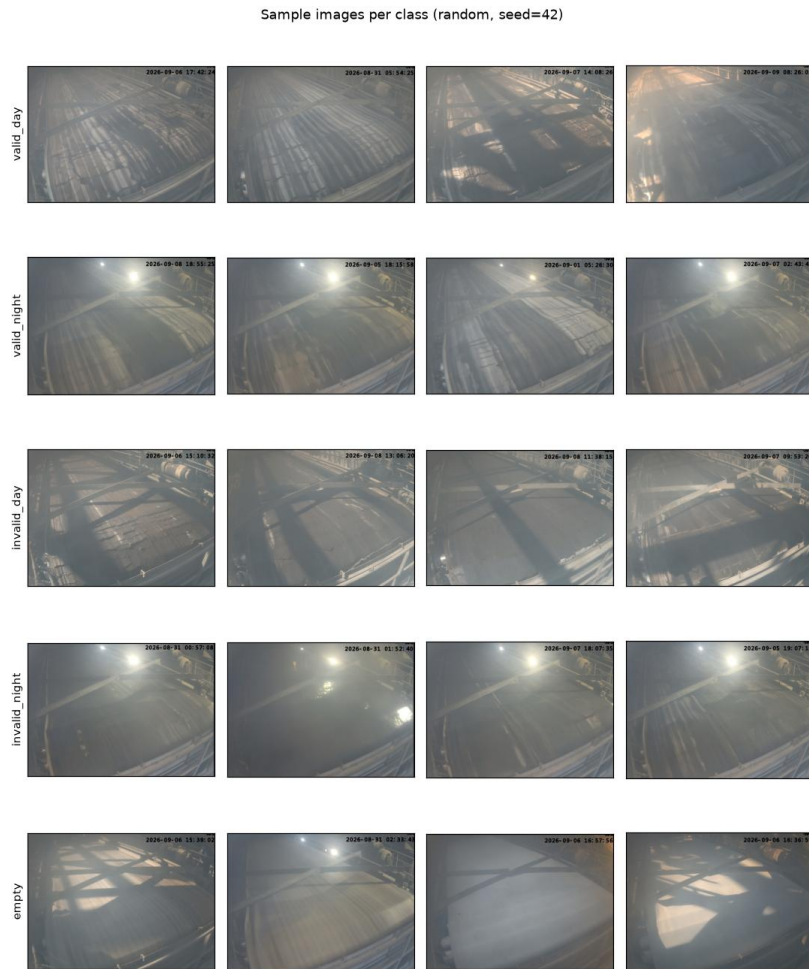


Figure 2. Random sample images per class (seed = 42).

3.3 Known Dataset Characteristics

- Class imbalance — “empty filter” has only 8 images vs. 30–62 for the other classes.
- Burned-in timestamp overlay — every image has a date/time stamp in the top-right corner, masked out before training so the model cannot key off it.
- Steam/dust obstruction — the process itself produces mist that partially obscures the filter in many shots; this is signal the model needs to be robust to, not noise to remove.
- Fixed camera angle — all images come from the same mounted camera, which simplifies the problem but means the model may be specific to this one installation.
- Source quality — images are WhatsApp exports (recompressed JPEGs), not original camera resolution.

3.4 The Day/Night Anomaly

Cross-referencing the manifest's recovered original capture timestamps against the day/night folder each photo came from turned up something unexpected: day- and night-labeled photos are interleaved within the same few seconds of each other, not hours apart. Example, from 2026-09-09:

```
11:29:43  day
11:29:47  night
11:29:49  day
```

11:29:50 day
11:29:56 night

All around 11:29 AM — clearly not solar daytime vs. nighttime. Whatever these two folders actually distinguish (two camera angles, flash vs. no flash, or a labeling judgment call made while manually sorting), it is not literal time-of-day.

This was checked a second way in Section 3.5's exploratory analysis, with the same conclusion. It remains an open question (Section 9).

3.5 Exploratory Data Analysis

A standalone script (eda.py) examined the raw dataset directly — not model outputs — producing the class-distribution and sample-grid figures above, plus a brightness comparison and an image-size check.

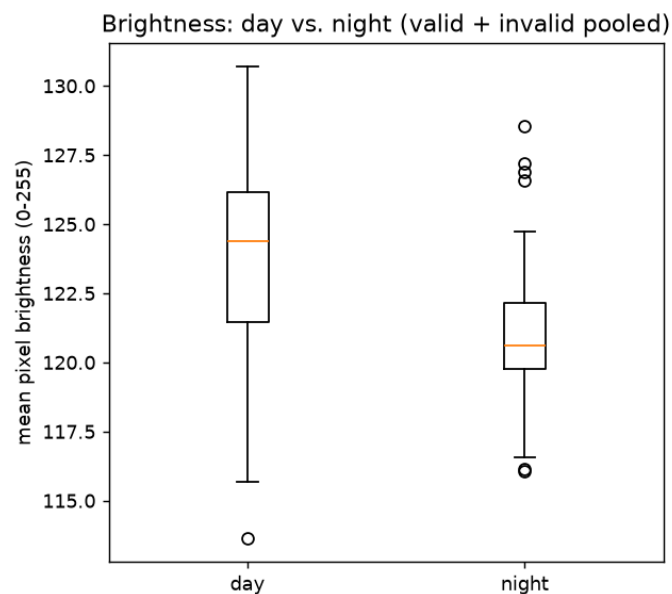


Figure 3. Mean per-image pixel brightness, day vs. night (valid + invalid pooled).

- Brightness finding: day and night are barely different on this measure — 123.6 (day, n=104) vs. 121.0 (night, n=64) mean brightness, heavily overlapping distributions. This is a second, independent line of evidence that whatever “day”/“night” actually encodes, it is not a simple global brightness difference — the floodlight's effect is a local hot spot, not a shift in the whole frame's average tone.
- Image size check: no two images in the dataset share identical pixel dimensions (176 distinct sizes, all within a narrow band around $\sim 1150\text{--}1185 \times 825\text{--}875$ pixels). This confirms the WhatsApp re-encoding theory and confirms that resizing every image to a consistent resolution during preprocessing is necessary, not just conventional.

4. Methodology

4.1 Preprocessing

- Timestamp overlay masked out (top 10% of the frame) so the model cannot key off the burned-in clock.
- Images resized to 240×240 then center-cropped (eval) or randomly cropped (train) to 224×224, the standard ImageNet input size, then normalized to ImageNet mean/std.
- Train-time augmentation: color jitter (brightness/contrast/saturation), small random rotation ($\pm 6^\circ$), and random crop — no horizontal flip, since the camera's framing (motor always upper-right) is not mirror-symmetric in real footage.

4.2 Leakage-Safe Train/Test Splitting

Photos taken seconds apart are near-duplicates. Splitting by individual image would let near-identical frames end up in both train and test, inflating the apparent accuracy. Instead, the manifest's recovered original capture timestamps were used to group photos into “sessions” (photos within 120 seconds of each other), and splits were made at the session level — an entire session goes to train or to test, never split across both.

- Three split ratios were tested for every label scheme: 80/20, 75/25, and 70/30.
- K-fold cross-validation was also run per scheme, with K auto-capped to whatever the scarcest class's independent session count actually supports (this varied from 2 to 3 depending on scheme — not every scheme could support the same K).
- The scarce “empty” class was floored at ≥ 3 training images regardless of split ratio.

4.3 Model Architecture and Fine-Tuning Strategy

Every model in this project uses the same architecture — MobileNetV2, pretrained on ImageNet — so that differences in results are attributable to the label scheme and split, not to architecture changes. MobileNetV2 consists of 19 feature blocks (1 stem convolution, 17 inverted-residual bottleneck blocks, and 1 head convolution), 52 convolutional layers in total, followed by a classifier head.

Only the last block (the head convolution) and a newly-added classifier layer (Linear(1280 $\rightarrow$ num_classes)) are unfrozen and trained; everything earlier stays frozen at its pretrained weights. This choice is dataset-size-driven, not a fixed rule:

- Overfitting risk: MobileNetV2 has $\sim 3.4\text{M}$ parameters. Fully fine-tuning all of it on 176 images (some classes as few as 8) would let the network memorize the training set instead of generalizing. Freezing most of the network acts as a strong regularizer.
- Early/mid layers are already general-purpose: pretrained on 1.2M ImageNet images, they have learned edges, textures, and shapes that transfer to almost any image task, this one included.
- Only the last block is specific enough to be worth adapting: the deeper a layer sits, the more it encodes concepts specific to the original task. Unfreezing just the last block lets the model reshape its final representation toward “cake texture is even vs. cracked/patchy” without touching the general-purpose feature extractors underneath.
- Compute: this project trains 17 configurations (23 counting cross-validation folds individually), mostly on CPU — fewer trainable parameters means faster training across that many runs.

- This matches the related-work finding in Section 2.3: transfer learning with a mostly-frozen backbone is standard practice specifically because industrial defect-detection datasets are usually small. With a much larger database (Section 10.2), unfreezing more blocks would likely improve on today's results further.

4.4 Handling Class Imbalance

The “empty” class has only 8 of 176 images (4.5%). Two mitigations were tried:

- Class-weighted CrossEntropyLoss (inverse-frequency weighting) — used in every training run.
- WeightedRandomSampler (oversampling) on top of the weighted loss — tested once as a variant and found to make no measurable difference (identical metrics to the plain run). Given the loss weighting already existed, resampling the same ~6 empty training images more often did not add new information; more real empty photos would be needed for further improvement.

4.5 Training Configuration

One fixed configuration was held constant across every comparison in this project, so that only one variable (label scheme, split ratio, or technique) changes at a time:

Parameter	Value
Backbone	MobileNetV2 (ImageNet-pretrained)
Fine-tuning	Last block + new classifier only (Section 4.3)
Optimizer	Adam, lr = 1e-3
Epochs	10
Batch size	16
Loss	Class-weighted CrossEntropyLoss
Random seed	42 (fixed throughout)
Environment	Google Colab (GPU) for real runs; local CPU for smoke tests

5. Experiments and Results

This section walks through the project's experiments in the order they were run, each one building on the previous stage's findings. All cross-validation (“CV”) numbers pool every fold's held-out predictions before scoring, and are the most trustworthy read in every table below — single train/test splits use very small test sets (as few as 10–16 images) and should be read with caution.

5.1 Stage 1 — Baseline: Lite vs. Extended Label Scheme

The first comparison tested whether simplifying day/night into a single label (the “lite”, 3-class scheme: valid / invalid / empty) costs anything against keeping day and night as separate classes (the “extended”, 5-class scheme). The same fixed configuration (Section 4.5) was swept across all three split ratios plus cross-validation, and one oversampling variant.

Variant	Accuracy	Val Loss	Macro F1	Invalid Recall
Lite, 80/20	0.600	—	0.450	0.600
Lite, 75/25	0.787	0.494	0.444	0.600
Lite, 70/30	0.587	0.326	0.365	0.303
Lite, 3-fold CV	0.633 ± 0.095	0.449	0.569	0.424 ± 0.019
Lite, 80/20 + oversampling	0.600	0.480	0.450	0.600
Extended, 80/20	0.632	—	0.415	0.643
Extended, 75/25	0.722	0.408	0.460	0.643
Extended, 70/30	0.741	0.384	0.479	0.714
Extended, 2-fold CV	0.723 ± 0.005	0.572	0.650	0.639 ± 0.075

Findings:

- Extended (5-class) beat lite (3-class) on every split ratio and in cross-validation. The CV numbers are the clearest signal — extended 72.3% ± 0.5 vs. lite 63.3% ± 9.5 accuracy. Lite's much larger CV standard deviation means its single-split numbers are far less stable session-to-session.
- No split ratio was uniformly best for both schemes — with this little independent data, which scheme is used matters more than the exact ratio.
- The oversampling variant made no difference on lite 80/20 — identical metrics to the plain run.

5.2 Stage 2 — No-Empty Scheme (Round 2)

A fourth label scheme was then tried: redefining “empty filter” as a kind of invalid rather than its own class, on the reasoning that an empty filter is arguably a fault state too, not a neutral third option. The 8 empty-filter photos were manually reassigned to invalid-day or invalid-night (6 day, 2 night, judged by eye), and read at two granularities: collapsed (2-class: valid/invalid) and split (4-class: valid_day / valid_night / invalid_day / invalid_night).

Variant	Accuracy	Val Loss	Macro F1	Invalid Recall
No-empty collapsed, 80/20	0.909	0.570	0.895	0.875

No-empty collapsed, 75/25	0.792	0.536	0.722	0.875
No-empty collapsed, 70/30	0.480	0.643	0.381	0.086
No-empty collapsed, 3-fold CV	0.708 $\pm$ 0.143	0.492	0.684	0.577 $\pm$ 0.267
No-empty split, 80/20	0.938	0.431	0.909	0.923
No-empty split, 75/25	0.782	0.411	0.764	0.824
No-empty split, 70/30	0.764	0.442	0.742	0.765
No-empty split, 2-fold CV	0.774 $\pm$ 0.019	0.474	0.760	0.711 $\pm$ 0.054

Findings — the headline result of the whole optimization effort:

- No-empty split (4-class) beat every scheme tried in this project, on cross-validation: 77.4% $\pm$ 1.9 accuracy vs. the previous best, extended, at 72.3% $\pm$ 0.5. It is both higher and has a tighter spread.
- No-empty collapsed (2-class) is not trustworthy despite its high single-split numbers: its CV variance is huge ($\pm$ 14.3 points — one fold scored 80.0%, another only 50.7%), almost certainly because collapsing day/night here throws away a real signal that the split version keeps.
- The “empty = invalid” reframing appears to genuinely help, not just shuffle the same information around — likely because the model no longer has to learn a 3rd/5th class from only 6–8 examples, and “empty” and “invalid” probably share real visual similarity (bare or torn filter surface) that the model can exploit once they are merged.

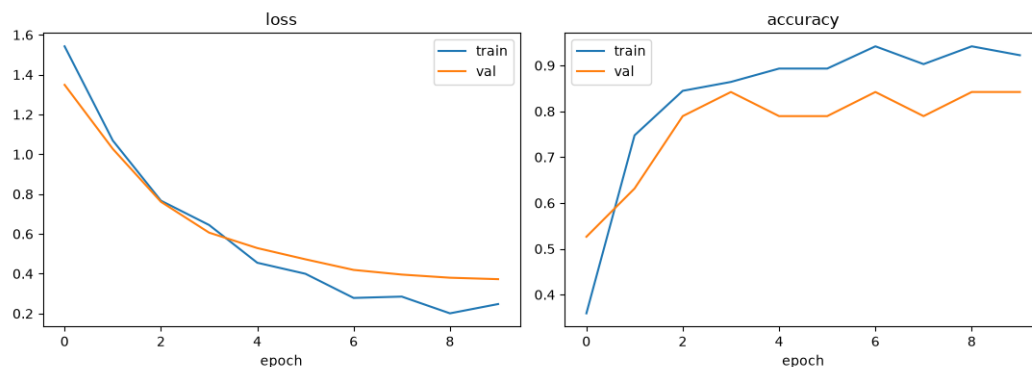


Figure 4. Training curves (loss and accuracy, train vs. val) for the extended 70/30 model.

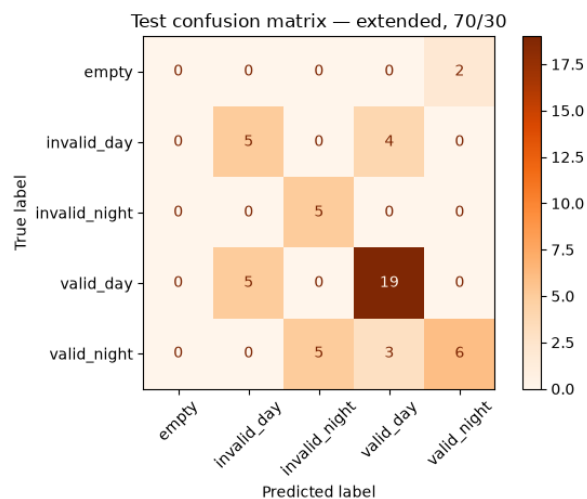


Figure 5. Test confusion matrix, extended 70/30 model.

5.3 Label-Scheme Decision

Section 5.1's cross-validation numbers put extended and no-empty split close together on one metric (ROC AUC, see Section 5.4), which initially looked like a genuine toss-up. The resolution came from recognizing a methodological flaw: that comparison scored each scheme against its own definition of “invalid” — a slightly different question per model. Putting both schemes on the same ground truth (the no-empty definition) in the same run (Section 5.5) resolved it: no-empty split won clearly on every metric — accuracy, invalid recall, ROC AUC (0.852 vs. 0.808), and PR AP (0.805 vs. 0.769).

Beyond the numbers, there is a stronger operational reason to fold “empty” into “invalid” rather than keep it as its own class, identified during review with the plant engineer:

1. The empty-filter dataset is tiny (8 of 176 images) — the statistical reason already covered above.
2. A live model does not need to detect “empty” at all — the operator empties the filter themselves, so they already know it is empty; the camera has no information advantage there. Asking the model to spend capacity distinguishing “empty” from “invalid” is solving a problem the operator does not have.
3. The natural production design follows from this: the model gets toggled off when the operator empties the filter, and toggled back on once filtration resumes — “empty” is an operator-controlled state to route around, not a class the vision model needs to recognize on its own.

Decision: the no-empty split (4-class) scheme — valid_day / valid_night / invalid_day / invalid_night — is the project's chosen scheme. Lite and no-empty collapsed were ruled out earlier by every analysis. This is the best-supported hypothesis given everything tried, not a production-validated result: the dataset is still 176 photos with a single random seed, run mostly on CPU.

5.4 Stage 3 — Threshold, ROC, PR, and Calibration Analysis

Every training run up to this point decided valid-vs-invalid with a bare `argmax()` — an implicit 0.5 probability cutoff that was never chosen deliberately or checked. All 23 individual trainings across the project (every ratio split × every cross-validation fold, all 4 schemes) were re-analyzed with full class probabilities instead of just the winning class, computing a ROC curve and AUC, a precision-recall curve and average precision (AP), a calibration curve, and two candidate best thresholds for flagging “invalid”: Youden's J (balances both error types) and an F1-optimal threshold (tuned specifically for the invalid class).

Config (CV = pooled)	ROC AUC	PR AP	Youden Threshold	F1-Optimal Threshold
Lite, CV	0.763	0.653	0.178	0.109
Extended, CV	0.837	0.752	0.302	0.302
No-empty collapsed, CV	0.723	0.720	0.526	0.168
No-empty split, CV	0.818	0.768	0.466	0.466

Findings:

- The default 0.5 cutoff has been under-flagging real faults the whole time. Nearly every config's optimal threshold, by either method, is below 0.5. Switching from the untuned default to a tuned

threshold would flag more real invalid cases as invalid, at the cost of a few more false alarms — a trade worth making, since a missed invalid costs more than a false alarm.

- On pure invalid-vs-not ranking quality (ROC AUC), extended edges out no-empty split (0.837 vs. 0.818) — the reverse of the accuracy-based ranking in Section 5.2. This is not a contradiction: accuracy measures the whole multi-class decision, while AUC measures specifically how well the model ranks “invalid” over “not invalid” regardless of where the cutoff sits. This is the toss-up resolved in Section 5.3.
- Lite and no-empty collapsed are weaker on both accuracy and ranking quality — consistent evidence across two different kinds of analysis that they are the two schemes to rule out first.

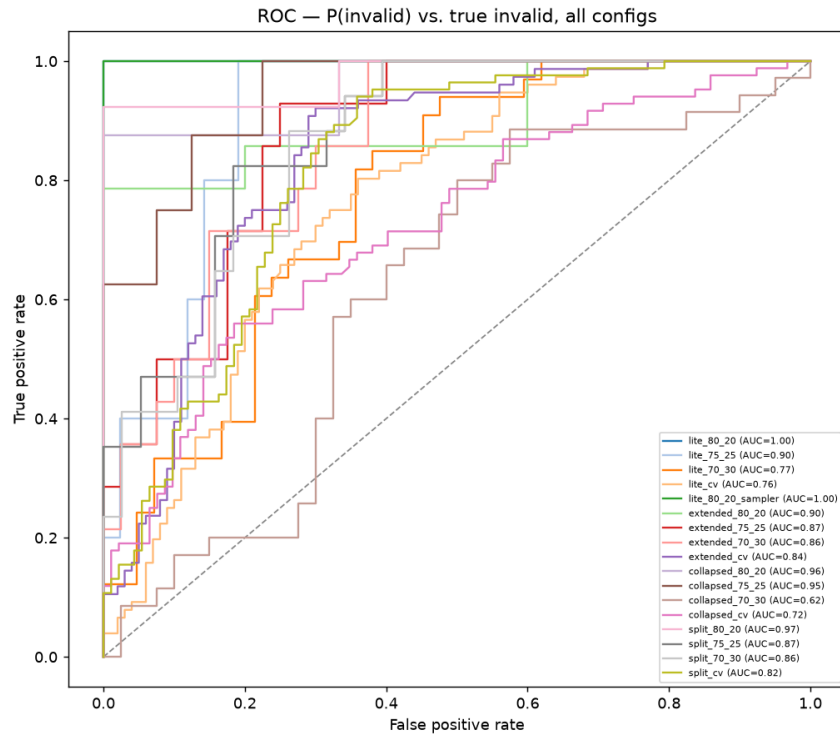


Figure 6. ROC curves, all 17 individual training configurations.

5.5 Stage 4 — Ensemble, Test-Time Augmentation, and Calibration

Three ways to push further were tried on the two closest competitors from Section 5.4 (extended CV vs. no-empty split CV): (1) averaging the two schemes' P(invalid) per image, each from its own properly held-out cross-validation fold model; (2) test-time augmentation (TTA), averaging the softmax over 6 mildly-augmented views per image at inference instead of one center crop; and (3) Platt-scaling calibration on top of the ensemble+TTA score. Ground truth throughout is the no-empty definition of “invalid” for every row, which is why this table puts both schemes on the same fair comparison referenced in Section 5.3.

Variant	ROC AUC	PR AP
Extended alone	0.808	0.769
No-empty split alone	0.852	0.805
Ensemble (extended + split, averaged)	0.843	0.803
Ensemble + TTA	0.847	0.787

Ensemble + TTA + calibration	0.847	0.787
------------------------------	-------	-------

Finding, reported straight, not spun: none of the three techniques beat the single best model. No-empty split alone (0.852 AUC / 0.805 AP) outperforms every ensemble variant. A plain 50/50 average pulled the stronger model down toward the weaker one rather than lifting it; TTA recovered a little of that on AUC but not on AP; calibration, as expected for a monotonic rescaling, left both ranking metrics unchanged, only improving probability quality, not discrimination. This is a useful negative result: with two models this close in quality but one clearly ahead, naive averaging is not automatically better than just using the better model. A confidence-weighted ensemble (weighting no-empty split more heavily than extended, rather than 50/50) is the natural next thing to try; it was not implemented here.

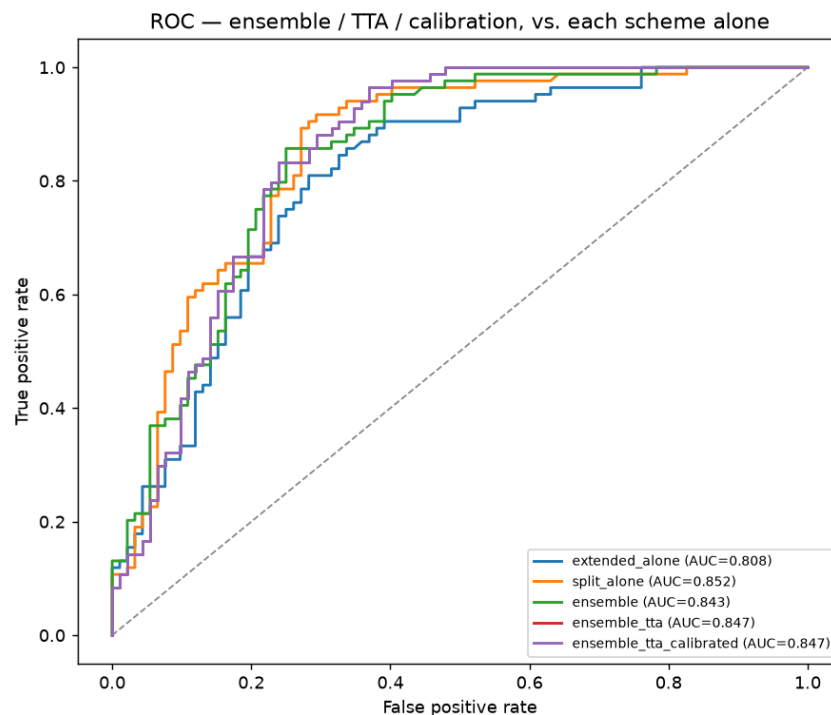


Figure 7. ROC curves — ensemble/TTA/calibration variants vs. each model alone.

5.6 Stage 5 — Live Demo Scripts

Two local scripts were built to demonstrate the classifier working end-to-end, both reusing the exact same preprocessing and model-loading code so there is no drift between what was evaluated and what runs live.

- `predict.py` — classifies one photo from the command line, printing the predicted class, confidence, and an operator-facing VALID/INVALID call (an “empty” prediction counts as INVALID, per the Section 5.3 decision). It automatically prefers the promoted no-empty-split model if it exists, falling back to the extended model (the only one actually trained for real so far) otherwise. Spot-checked against three photos: a valid_day photo (89% confidence, correct), an invalid_night photo (98% confidence, correct), and a valid_night photo that came back a lower-confidence 47% valid_night with empty and invalid_night close behind — consistent with the documented finding that “empty” is this model's weakest class, not a bug in the script.
- `watch_camera_feed.py` — a simulated live feed: watches a folder for new photos and classifies each one within a few seconds of it appearing, printing an alert line whenever the call is INVALID. This is

explicitly a simulation, not a real camera integration — a real deployment would need an actual frame source (an RTSP stream, or the camera's own image export) and a real alerting channel (SMS, dashboard, or log file), neither of which is decided yet. Tested locally with a valid and an invalid sample photo; both were classified correctly and the alert fired correctly on the invalid one.

6. Decision Log

A consolidated record of every major decision made over the course of the project, in order, with the reasoning behind each one.

#	Decision	Reasoning
1	Framework: PyTorch + transfer learning (MobileNetV2)	Small dataset (176 images) cannot support training a CNN from scratch; transfer learning is standard practice for this data regime (Section 2.3).
2	Only the last backbone block + classifier are fine-tuned	Overfitting risk on 176 images vs. 3.4M parameters; early/mid layers are already general-purpose (Section 4.3).
3	Leakage-safe splitting by capture session, not by image	Photos seconds apart are near-duplicates; per-image splitting would leak information across train/test.
4	Class-weighted loss for the imbalanced “empty” class	Standard mitigation for imbalance; a WeightedRandomSampler variant was also tried and found to add nothing further.
5	Extended (5-class) scheme preferred over lite (3-class)	Extended beat lite on every split ratio and in cross-validation (Section 5.1).
6	No-empty split (4-class) adopted as the final label scheme	Wins on every metric once both schemes are compared against the same ground truth (Section 5.3), plus a strong operational argument: a live model does not need to detect “empty” at all.
7	Decision threshold tuned away from the naive 0.5 default	Every config's optimal threshold (Youden's J or F1-optimal) was found to sit below 0.5, meaning the default was under-flagging real faults (Section 5.4).
8	Ensembling not adopted	None of ensemble / TTA / calibration beat the single best model — an honest negative result (Section 5.5).
9	Live demo built as local scripts, not a full deployment	Real camera integration and alerting channel are still open decisions (Section 9); the scripts demonstrate the classifier works end-to-end in the meantime.
10	Repository made public	Satisfies the course submission requirement for a public repository link.

7. Business Case and Value Estimate

This section works through the monetized value of automating filtration-quality detection, with every assumption stated explicitly so it can be corrected with the plant's real operating figures.

7.1 Yield Loss

Quantity	Value
Base rate	\$2,000 / hour per 1% P ₂ O ₅ remaining in gypsum
Typical equivalent	~3% → ~\$6,000 / hour effective rate during a bad-filtration episode
Affected operating time	~6% of operating hours, on an assumed ~8,000 hr/year (~91% uptime) → ~480 hr/year
Annual yield-loss exposure	~\$2,880,000 / year

7.2 Downtime (Equipment Damage)

Quantity	Value
Lost-production rate during downtime	\$49,500 / hour (45 × 1,100, per the plant engineer)
Additional annual downtime	~3% of ~8,000 hr/year → ~240 hr/year
Annual downtime exposure	~\$11,880,000 / year

7.3 Combined Value Estimate

Quantity	Value
Total annual exposure (yield loss + downtime)	~\$14,760,000 / year
Automated-detection recovery, at 10–15%	~\$1,476,000 – \$2,214,000 / year

The 10–15% recovery estimate reflects catching bad filtration sooner and more consistently than intermittent manual checks can — the gap between when a problem happens and when it is noticed, which this project's classifier is meant to close. Every assumed figure above (annual operating hours, percentage of hours affected) can be replaced with the plant's real operating data to sharpen this estimate.

8. Conclusions

- No-empty split (4-class) is the strongest model found: ~77% cross-validation accuracy, ROC AUC 0.852 — clearly ahead of every other scheme once compared on the same ground truth.
- Folding “empty filter” into “invalid” measurably helps, and it is the operationally correct design, not just a statistical convenience: the empty class is tiny (8 images), and a live deployment does not need to detect “empty” at all, since the operator empties the filter themselves and already knows it.
- The naive 0.5 probability cutoff was under-flagging real faults — tuned thresholds (~0.3–0.5, F1-optimal) catch more actual invalid cases at a small cost in false alarms.
- Ensembling, test-time augmentation, and calibration did not beat the single best model — reported as a straight negative result rather than spun.
- Automated detection is estimated to recover ~\$1.48M–\$2.21M/year, at 10–15% recovery of the ~\$14.76M/year combined exposure (yield loss + downtime from equipment damage).

9. Limitations

- Small dataset, by IT constraint, not by choice — access to plant photos for this project was limited by IT/data-access restrictions, leaving only 176 images (8 of them “empty”). The results here are real and better than chance would predict, but every accuracy number carries meaningful uncertainty (cross-validation standard deviations of several points).
- Single random seed, mostly CPU-trained runs — results could shift with a different seed or backbone.
- The promoted model has not been trained on a real, full (non-abbreviated) run yet — the consolidated notebook still needs a real pass in Colab to produce final weights; “promoted” is provisional.
- Day/night remains unexplained — literal time-of-day and whole-image brightness have both been ruled out, but the real distinguishing factor is still unknown.
- The live-feed demo is a simulation (folder-watcher + console alert), not a real camera/RTSP integration or real alerting channel.
- Single fixed camera install, uncontrolled lighting — unclear how well this generalizes to a different angle or to genuinely fixed lighting without retraining.
- Source images are WhatsApp-recompressed, not native camera resolution.
- The model currently only outputs a binary valid/invalid call — no sense of how valid or invalid, or of trend over time.

10. Further Directions

10.1 Near-Term

- Run the consolidated training notebook for real to get trustworthy final numbers and the actual promoted model weights.
- Try confidence-weighted ensembling (weighting the stronger model more) instead of a plain average.
- Build the real camera feed and alerting channel, replacing the folder-watcher simulation.

10.2 Full-Scale Pilot Roadmap

Proposed by the plant engineer during project review:

- Retrain on a much larger database once IT/data-access constraints are lifted — the current 176-image result is a proof of concept, not the ceiling.
- Unfreeze more of the backbone — the current fine-tuning strategy (only the last MobileNetV2 block + classifier trainable) is a dataset-size-driven choice, not a permanent one; a much larger database would support unfreezing more blocks, or full fine-tuning, likely improving on today's results further.
- Add projectors around the filter to normalize lighting between “day” and “night” conditions and reduce shadow effects — engineers the day/night ambiguity away at the source instead of continuing to try to explain it after the fact.
- Move beyond binary valid/invalid to a continuous filtration-quality score, so the process's actual trend is visible, not just a pass/fail flag.
- Make it a time-based model that tracks change over a sequence of frames and flags when the process is starting to turn bad — early warning instead of single-frame classification after the fact.
- Far future: a downstream model that calculates the right reactor process parameters based on the predicted filtration state — closing the loop from vision straight through to process control, not just alerting an operator.

11. Appendix

A. Repository Structure

Code repository: github.com/yaronconroy-del/picture-recognition-gypsum (public)

- Model & Training/pictures/ — the dataset, in three parallel folder structures (extended 5-class, simple 3-class, no-empty 4-class).
- Model & Training/eda/ — exploratory data analysis outputs (Section 3.5).
- Model & Training/scripts/ — all Python scripts: dataset indexing, every training experiment, the EDA script, and the two live-demo scripts.
- Model & Training/notebooks/ — the Colab notebooks, including full_pipeline.ipynb, the single consolidated notebook reproducing every experiment in this report.
- Model & Training/models/ — trained model weights and every experiment's result files and plots.
- Evaluation & Docs/Project Workflow.md — the full chronological project log this report was built from, including additional detail not reproduced here.
- Evaluation & Docs/Final Presentation.pptx / .pdf — the course submission slide deck.

B. References

A Systematic Review on Deep Learning with CNNs Applied to Surface Defect Detection. PMC10607335. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10607335/>

Tackling class imbalance in computer vision: a contemporary review. Artificial Intelligence Review, Springer Nature. <https://link.springer.com/article/10.1007/s10462-023-10557-6>

A Deep Transfer Learning-Based Visual Inspection System for Assembly Defects in Similar Types of Manual Tool Products. PMC11945499. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11945499/>